Animixel

0.0.1

Generated by Doxygen 1.18.0

1 Data Structure Index 1

1.1 Data Structures . . . 1

2 File Index 3

2.1 File List . . . 3

3 Data Structure Documentation 5

3.1 Custom_Element_Data::[union].__unnamed0__ Union Reference . . . 5

3.2 Canvas Struct Reference . . . 5

3.2.1 Detailed Description . . . 6

3.3 Custom_Element_Data Struct Reference . . . 6

3.3.1 Detailed Description . . . 6

3.4 Input_Handler Struct Reference . . . 6

3.4.1 Detailed Description . . . 7

4 File Documentation 9

4.1 src/canvas.h File Reference . . . 9

4.1.1 Detailed Description . . . 10

4.1.2 Function Documentation . . . 10

4.1.2.1 canvas_bresenham() . . . 10

4.1.2.2 canvas_clear_pixel() . . . 10

4.1.2.3 canvas_draw() . . . 11

4.1.2.4 canvas_dump_set() . . . 11

4.1.2.5 canvas_free() . . . 11

4.1.2.6 canvas_init() . . . 12

4.1.2.7 canvas_print() . . . 12

4.1.2.8 canvas_set_pixel() . . . 12

4.2 canvas.h . . . 13

4.3 src/input.h File Reference . . . 13

4.3.1 Detailed Description . . . 14

4.3.2 Enumeration Type Documentation . . . 14

4.3.2.1 keystate . . . 14

4.3.3 Function Documentation . . . 15

4.3.3.1 cursor_in_bounds() . . . 15

4.3.3.2 input_handler_init() . . . 15

4.3.3.3 update_key_state() . . . 16

4.4 input.h . . . 16

4.5 src/math_utils.h File Reference . . . 17

4.5.1 Detailed Description . . . 17

4.5.2 Function Documentation . . . 17

4.5.2.1 clampi() . . . 17

4.5.2.2 clampui() . . . 18

4.6 math_utils.h . . . 18

4.7 src/ui_core.h File Reference 18
4.7.1 Detailed Description 19
4.7.2 Function Documentation 19
4.7.2.1 clay_init() 19
4.7.2.2 clay_render() 20
4.7.2.3 clay_update_dimensions() 20
4.7.2.4 create_static_button() 20
4.7.2.5 editor_layout() 21
4.8 ui_core.h 21

Index **23**

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

Custom_Element_Data::[union].__unnamed0__ 5
Canvas
Struct that describes a drawing canvas 5
Custom_Element_Data
Struct used to hold the data of a Clay custom element as a tagged union 6
Input_Handler
Struct that acts as input handling interface 6

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/canvas.h
: Contains the definition for a Canvas and its associated functions 9

src/input.h
: Contains structs/functions for handling user input 13

src/math_utils.h
: Contains the definition various mathematical helper functions/values 17

src/ui_core.h
: Contains structs/functions for UI using Clay 18

Chapter 3

Data Structure Documentation

3.1 Custom_Element_Data::[union].__unnamed0__ Union Reference

Data Fields

- Canvas * **canvas**

  Tagged union pointer element for a canvas.

The documentation for this union was generated from the following files:

3.2 Canvas Struct Reference

Struct that describes a drawing canvas.

`#include <canvas.h>`

Data Fields

- uint32_t * **pixels**

  Array holding the pixel data.
- BOB_Pixelbuffer_Handle **pb**

  Reference handle to the pixelbuffer that actually renders the canvas data to the screen.
- size_t **width**

  Width of the canvas.
- size_t **height**

  Height of the canvas.
- size_t **scale**

  Scale factor of the canvas. Used when rendering.
- uint32_t **origin_x**

  Top left x coord.
- uint32_t **origin_y**

  Top left y coord.

3.2.1 Detailed Description

Struct that describes a drawing canvas.

The documentation for this struct was generated from the following file:

- src/canvas.h

3.3 Custom_Element_Data Struct Reference

Struct used to hold the data of a Clay custom element as a tagged union.

`#include <ui_core.h>`

Data Structures

- union [union].__unnamed0__

Data Fields

- Custom_Element_Type **type**

  Tag that indicates the type of the custom element data.
- union

3.3.1 Detailed Description

Struct used to hold the data of a Clay custom element as a tagged union.

All of the union elements are pointers to maintain a consistent struct size

The documentation for this struct was generated from the following file:

- src/ui_core.h

3.4 Input_Handler Struct Reference

Struct that acts as input handling interface.

`#include <input.h>`

Data Fields

- keystate **key_status** [NUM_KEYS]

  Map that holds the state of all the current keys we care about.
- double **mouse_x**

  Mouse x position in the current frame.
- double **mouse_y**

  Mouse y position in the current frame.
- double **prev_x**

  Mouse x position in the previous frame.
- double **prev_y**

  Mouse y position in the previous frame.
- double **scroll_x**

  Mouse horizontal scroll in the current frame.
- double **scroll_y**

  Mouse vertical scroll in the current frame.

3.4.1 Detailed Description

Struct that acts as input handling interface.

The documentation for this struct was generated from the following file:

- src/input.h

Chapter 4

File Documentation

4.1 src/canvas.h File Reference

Contains the definition for a Canvas and its associated functions.

```
#include <stdint.h>
#include <stddef.h>
#include <bob.h>
```

Data Structures

- struct Canvas

  Struct that describes a drawing canvas.

Functions

- void canvas_init (Canvas *c, size_t width, size_t height, size_t scale, uint32_t default_colour, BOB_↩ Renderer_Handle r)

  Initialises a Canvas.
- void canvas_free (Canvas *c)

  Frees all memory used by member variables of a Canvas outside of the struct itself, but does not destroy the memory region occupied by the struct.
- void canvas_draw (Canvas *c, BOB_Renderer_Handle renderer)

  Draws a canvas to the screen.
- void canvas_set_pixel (Canvas *c, uint32_t x, uint32_t y, uint32_t colour)

  Sets a pixel in the Canvas.
- void canvas_clear_pixel (Canvas *c, uint32_t x, uint32_t y)

  Erases a pixel in the Canvas by setting its RGBA colour to (0, 0, 0, 0).
- void canvas_bresenham (Canvas *c, uint32_t sx, uint32_t sy, uint32_t ex, uint32_t ey, uint32_t colour)

  Uses Bresenham's line algorithm adapted from here: https://www.geeksforgeeks.org/dsa/↩ bresenhams-line-generation-algorithm/ to draw a line between two points on a Canvas.
- void canvas_print (Canvas *c)

  Prints the pixels array of the Canvas to the terminal.
- void canvas_dump_set (Canvas *c)

  Prints the set pixels array of the Canvas to the terminal.

4.1.1 Detailed Description

Contains the definition for a Canvas and its associated functions.

4.1.2 Function Documentation

4.1.2.1 canvas_bresenham()

```
void canvas_bresenham (
            Canvas * c,
            uint32_t sx,
            uint32_t sy,
            uint32_t ex,
            uint32_t ey,
            uint32_t colour)
```

Uses Bresenham's line algorithm adapted from here: https://www.geeksforgeeks.org/dsa/bresenhams-line-generation-algorithm/ to draw a line between two points on a Canvas.

Parameters

c	a pointer to the Canvas where the line should be drawn
sx	the starting x coord of the line
sy	the starting y coord of the line
ex	the ending x coord of the line
ey	the ending y coord of the line
colour	the colour the line whould be drawn in

4.1.2.2 canvas_clear_pixel()

```
void canvas_clear_pixel (
            Canvas * c,
            uint32_t x,
            uint32_t y)
```

Erases a pixel in the Canvas by setting its RGBA colour to (0, 0, 0, 0).

Parameters

c	a pointer to the Canvas where the pixel should be erased
x	the x coordinate of the pixel
y	the y coordinate of the pixel

4.1.2.3 canvas_draw()

```
void canvas_draw (
            Canvas * c,
            BOB_Renderer_Handle renderer)
```

Draws a canvas to the screen.

Updates the Canvas' pixelbuffer with the new pixel values from the pixels array. Determines the size of the destination quad using the width, height and scale member variables

Parameters

c	a pointer to the Canvas to be drawn
r	a handle to the renderer that was used to create the Canvas' pixelbuffer

4.1.2.4 canvas_dump_set()

```
void canvas_dump_set (
            Canvas * c)
```

Prints the set pixels array of the Canvas to the terminal.

Used for debugging and should not be used in production code. Preffered over canvas_print for large sizes where printing the full Canvas would be unreadable

Parameters

c	a pointer to the Canvas whose set pixels should be printed

4.1.2.5 canvas_free()

```
void canvas_free (
            Canvas * c)
```

Frees all memory used by member variables of a Canvas outside of the struct itself, but does not destroy the memory region occupied by the struct.

Parameters

c	a pointer to the Canvas whole members must be freed

4.1.2.6 canvas_init()

```
void canvas_init (
            Canvas * c,
            size_t width,
            size_t height,
            size_t scale,
            uint32_t default_colour,
            BOB_Renderer_Handle r)
```

Initialises a Canvas.

Does not allocate memory for the struct itself (this must be done beforehand using either a local struct variable or a heap allocation), but will allocate memory for the pixels buffer and will create the PixelBuffer via a call to BOB

Parameters

c	a pointer to a Canvas to be initialised
width	the desired width of the Canvas
height	the desired height of the Canvas
scale	the desired scale factor of the Canvas
default_colour	the background colour the Canvas should default to when first drawn
r	the handle of the BOB_Renderer the Canvas should use to generate its pixelbuffer

4.1.2.7 canvas_print()

```
void canvas_print (
            Canvas * c)
```

Prints the pixels array of the Canvas to the terminal.

Used for debugging and should not be used in production code

Parameters

c	a pointer to the Canvas whose pixels should be printed

4.1.2.8 canvas_set_pixel()

```
void canvas_set_pixel (
            Canvas * c,
            uint32_t x,
            uint32_t y,
            uint32_t colour)
```

Sets a pixel in the Canvas.

Only updates the pixels array, not the pixelbuffer

Parameters

c	a pointer to the Canvas where the pixel should be set
x	the x coordinate of the pixel
y	the y coordinate of the pixel
colour	the new colour the pixel should be set to

4.2 canvas.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef CANVAS_H
00007 #define CANVAS_H
00008
00009 #include <stdint.h>
00010 #include <stddef.h>
00011 #include <bob.h>
00012
00014 typedef struct {
00015     uint32_t *pixels;
00016     BOB_Pixelbuffer_Handle pb;
00017     size_t width;
00018     size_t height;
00019     size_t scale;
00020     uint32_t origin_x;
00021     uint32_t origin_y;
00022 } Canvas;
00023
00035 void canvas_init(Canvas *c, size_t width, size_t height, size_t scale, uint32_t default_colour,
      BOB_Renderer_Handle r);
00041 void canvas_free(Canvas *c);
00048 void canvas_draw(Canvas *c, BOB_Renderer_Handle renderer);
00056 void canvas_set_pixel(Canvas *c, uint32_t x, uint32_t y, uint32_t colour);
00063 void canvas_clear_pixel(Canvas *c, uint32_t x, uint32_t y);
00074 void canvas_bresenham(Canvas *c, uint32_t sx, uint32_t sy, uint32_t ex, uint32_t ey, uint32_t colour);
00080 void canvas_print(Canvas *c);
00087 void canvas_dump_set(Canvas *c);
00088
00089 #endif
```

4.3 src/input.h File Reference

Contains structs/functions for handling user input.

Data Structures

- struct Input_Handler

  Struct that acts as input handling interface.

Enumerations

- enum keystate { KEY_JUST_PRESSED , KEY_PRESSED , KEY_RELEASED , KEY_JUST_RELEASED }

  Enum to hold the state of the keys.

- enum keys {
  KEY_0 , **KEY_1** , **KEY_2** , **KEY_3** ,
  KEY_4 , **KEY_5** , **KEY_6** , **KEY_7** ,
  KEY_8 , **KEY_9** , **KEY_A** , **KEY_B** ,
  KEY_C , **KEY_D** , **KEY_E** , **KEY_F** ,
  KEY_G , **KEY_H** , **KEY_I** , **KEY_J** ,
  KEY_K , **KEY_L** , **KEY_M** , **KEY_N** ,
  KEY_O , **KEY_P** , **KEY_Q** , **KEY_R** ,
  KEY_S , **KEY_T** , **KEY_U** , **KEY_V** ,
  KEY_W , **KEY_X** , **KEY_Y** , **KEY_Z** ,
  KEY_MINUS , **KEY_PLUS** , **MOUSE_BUTTON_LEFT** , **MOUSE_BUTTON_RIGHT** ,
  MOUSE_BUTTON_MIDDLE , **NUM_KEYS** }

  Enum that holds all of the keys we currently want to work with.

Functions

- void input_handler_init (Input_Handler *ih)

  Initialises an Input_Handler.
- void update_key_state (Input_Handler *ih)

  Updates the key state after user input processing.
- int cursor_in_bounds (Input_Handler *ih, float min_x, float min_y, float max_x, float max_y)

  Simple checker that returns if the cursor is in the bounds [min_x, max_x), [min_y, max_y).

4.3.1 Detailed Description

Contains structs/functions for handling user input.

4.3.2 Enumeration Type Documentation

4.3.2.1 keystate

```
enum keystate
```

Enum to hold the state of the keys.

Enumerator

KEY_JUST_PRESSED	Was pressed this frame.
KEY_PRESSED	Held on from the previous frame.
KEY_RELEASED	Not pressed.
KEY_JUST_RELEASED	Released the previous frame.

4.3.3 Function Documentation

4.3.3.1 cursor_in_bounds()

```
int cursor_in_bounds (
            Input_Handler * ih,
            float min_x,
            float min_y,
            float max_x,
            float max_y)
```

Simple checker that returns if the cursor is in the bounds [min_x, max_x), [min_y, max_y).

Parameters

ih	a pointer to the Input_Handler whose mouse position we are checking
min_x	the minimum x value the cursor can hold
min_y	the minimum y value the cursor can hold
max_x	the maximum x value the cursor can hold
max_y	the maximum y value the cursor can hold

Returns

1 if the cursor is in the bounds of the window, 0 otherwise

4.3.3.2 input_handler_init()

```
void input_handler_init (
            Input_Handler * ih)
```

Initialises an Input_Handler.

Sets the initial cursor position to (0, 0) and all of the keys to KEY_RELEASED Does not allocate memory for the Input_Handler, this must be done before the function was called

Parameters

ih	a pointer to the Input_Handler to be initialised

4.3.3.3 update_key_state()

```
void update_key_state (
            Input_Handler * ih)
```

Updates the key state after user input processing.

Parameters

ih	a pointer to the Input_Handler whose key map needs updating

4.4 input.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef __INPUT_H__
00007 #define __INPUT_H__
00008
00010 typedef enum {
00011     KEY_JUST_PRESSED,
00012     KEY_PRESSED,
00013     KEY_RELEASED,
00014     KEY_JUST_RELEASED
00015 } keystate;
00016
00018 typedef enum {
00019     KEY_0,
00020     KEY_1,
00021     KEY_2,
00022     KEY_3,
00023     KEY_4,
00024     KEY_5,
00025     KEY_6,
00026     KEY_7,
00027     KEY_8,
00028     KEY_9,
00029     KEY_A,
00030     KEY_B,
00031     KEY_C,
00032     KEY_D,
00033     KEY_E,
00034     KEY_F,
00035     KEY_G,
00036     KEY_H,
00037     KEY_I,
00038     KEY_J,
00039     KEY_K,
00040     KEY_L,
00041     KEY_M,
00042     KEY_N,
00043     KEY_O,
00044     KEY_P,
00045     KEY_Q,
00046     KEY_R,
00047     KEY_S,
00048     KEY_T,
00049     KEY_U,
00050     KEY_V,
00051     KEY_W,
00052     KEY_X,
00053     KEY_Y,
00054     KEY_Z,
00055     KEY_MINUS,
00056     KEY_PLUS,
00057     MOUSE_BUTTON_LEFT,
00058     MOUSE_BUTTON_RIGHT,
00059     MOUSE_BUTTON_MIDDLE,
00060     NUM_KEYS
00061 } keys;
00062
00064 typedef struct {
00065     keystate key_status[NUM_KEYS];
00066     double mouse_x;
00067     double mouse_y;
00068     double prev_x;
00069     double prev_y;
```

```
00070     double scroll_x;
00071     double scroll_y;
00072 } Input_Handler;
00073
00080 void input_handler_init(Input_Handler *ih);
00085 void update_key_state(Input_Handler *ih);
00095 int cursor_in_bounds(Input_Handler *ih, float min_x, float min_y, float max_x, float max_y);
00096
00097 #endif
```

4.5 src/math_utils.h File Reference

Contains the definition various mathematical helper functions/values.

```
#include <stdint.h>
```

Functions

- int32_t clampi (int32_t val, int32_t min, int32_t max)

  Clamps a signed integer between two bounds (inclusive).
- uint32_t clampui (uint32_t val, uint32_t min, uint32_t max)

  Clamps an unsigned integer between two bounds (inclusive).

4.5.1 Detailed Description

Contains the definition various mathematical helper functions/values.

4.5.2 Function Documentation

4.5.2.1 clampi()

```
int32_t clampi (
            int32_t val,
            int32_t min,
            int32_t max)
```

Clamps a signed integer between two bounds (inclusive).

Parameters

val	the integer to be clamped
min	the lower bound
max	the upper bound

Returns

the clamped value of val

4.5.2.2 clampui()

```
uint32_t clampui (
            uint32_t val,
            uint32_t min,
            uint32_t max)
```

Clamps an unsigned integer between two bounds (inclusive).

Parameters

val	the integer to be clamped
min	the lower bound
max	the upper bound

Returns

the clamped value of val

4.6 math_utils.h

Go to the documentation of this file.

```
00001
00005 #ifndef MATH_UTILS_H
00006 #define MATH_UTILS_H
00007
00008 #include <stdint.h>
00009
00017 int32_t clampi(int32_t val, int32_t min, int32_t max);
00025 uint32_t clampui(uint32_t val, uint32_t min, uint32_t max);
00026
00027 #endif // MATH_UTILS_H
```

4.7 src/ui_core.h File Reference

Contains structs/functions for UI using Clay.

```
#include <bob.h>
#include <clay.h>
#include "canvas.h"
#include "input.h"
```

Data Structures

- struct Custom_Element_Data

  Struct used to hold the data of a Clay custom element as a tagged union.

Enumerations

- enum Custom_Element_Type { **CUSTOM_ELEMENT_TYPE_CANVAS** }

  Enum used to tag different Clay Custom element types.

Functions

- void create_static_button (uint32_t id, uint32_t width, uint32_t height, uint8_t padding, Clay_Color hover_↩ colour, Clay_Color basic_colour, Clay_String text, void(∗button_handler)(Clay_ElementId id, Clay_Pointer↩ Data info, void ∗userData), void ∗userData)

  Creates a clay button widget whose size does not change.
- void clay_init (size_t width, size_t height)

  Initialises Clay.
- void clay_update_dimensions (Input_Handler ∗ih, float dt, size_t width, size_t height)

  Updates the screen dimensions used by clay as well as mouse/scroll positions.
- void clay_render (BOB_Renderer_Handle r, Clay_RenderCommandArray renderCommands)

  Draws the Clay UI to the screen.
- void **clay_free** ()

  Destroys any resources used by clay during rendering.
- Clay_RenderCommandArray editor_layout (float dt, Custom_Element_Data ∗canvas)

  Defines the Clay layout for the main editor page.

Variables

- BOB_Font_Handle **bitmap**

  Global currently used to store the bitmap font used for text rendering.

4.7.1 Detailed Description

Contains structs/functions for UI using Clay.

4.7.2 Function Documentation

4.7.2.1 clay_init()

```
void clay_init (
            size_t width,
            size_t height)
```

Initialises Clay.

Parameters

width	the width of the region the Clay UI should occupy
height	the height of the region the Clay UI should occupy

4.7.2.2 clay_render()

```
void clay_render (
            BOB_Renderer_Handle r,
            Clay_RenderCommandArray renderCommands)
```

Draws the Clay UI to the screen.

Parameters

r	the handle of the BOB_Renderer used for drawing
renderCommands	an array containing all of the necessary draw commands

4.7.2.3 clay_update_dimensions()

```
void clay_update_dimensions (
            Input_Handler * ih,
            float dt,
            size_t width,
            size_t height)
```

Updates the screen dimensions used by clay as well as mouse/scroll positions.

Parameters

ih	a pointer to the input handler used to hold the new mouse/scroll positions
dt	the time elapsed since last frame (in milliseconds)
width	the new region width
height	the new region height

4.7.2.4 create_static_button()

```
void create_static_button (
            uint32_t id,
            uint32_t width,
            uint32_t height,
            uint8_t padding,
            Clay_Color hover_colour,
            Clay_Color basic_colour,
            Clay_String text,
            void(* button_handler )(Clay_ElementId id, Clay_PointerData info, void *userData),
            void * userData)
```

Creates a clay button widget whose size does not change.

Parameters

id	the unique numeric id of the button to differentiate it from other buttons
width	the width of the button

height	the height of the button
padding	the padding of the button
hover_colour	the colour the button should turn when the mouse hovers over it
basic_colour	the colour the button should stay when not hovered over
text	the text the button should contain
button_handler	callback function that should be called when the button is clicked
userData	the data that should be passed to the button_handler callback

4.7.2.5 editor_layout()

```
Clay_RenderCommandArray editor_layout (
            float dt,
            Custom_Element_Data * canvas)
```

Defines the Clay layout for the main editor page.

Parameters

dt	the time elapsed since last frame (in milliseconds)
canvas	a pointer to the Custom_Element_Data holding the render data for the canvas

Returns

a Clay_RenderCommandArray that holds all the draw commands for this layout

4.8 ui_core.h

Go to the documentation of this file.

```
00001
00005 #ifndef UI_CORE_H
00006 #define UI_CORE_H
00007
00008 #include <bob.h>
00009 #include <clay.h>
00010 #include "canvas.h"
00011 #include "input.h"
00012
00014 typedef enum {
00015     CUSTOM_ELEMENT_TYPE_CANVAS,
00016 } Custom_Element_Type;
00017
00021 typedef struct {
00022     Custom_Element_Type type;
00023     union {
00024         Canvas *canvas;
00025     };
00026 } Custom_Element_Data;
00027
00040 void create_static_button(uint32_t id, uint32_t width, uint32_t height, uint8_t padding, Clay_Color
      hover_colour,
00041                           Clay_Color basic_colour, Clay_String text, void
      (*button_handler)(Clay_ElementId id,
00042                           Clay_PointerData info, void *userData), void *userData);
00043
00045 extern BOB_Font_Handle bitmap; //TODO: Replace this with something else so we can use different font
      sizes for different widgets
00046
00052 void clay_init(size_t width, size_t height);
00060 void clay_update_dimensions(Input_Handler *ih, float dt, size_t width, size_t height);
00066 void clay_render(BOB_Renderer_Handle r, Clay_RenderCommandArray renderCommands);
00070 void clay_free();
00077 Clay_RenderCommandArray editor_layout(float dt, Custom_Element_Data *canvas);
00078
00079 #endif // UI_CORE_H
```

Index

Canvas, 5
canvas.h
    canvas_bresenham, 10
    canvas_clear_pixel, 10
    canvas_draw, 10
    canvas_dump_set, 11
    canvas_free, 11
    canvas_init, 11
    canvas_print, 12
    canvas_set_pixel, 12
canvas_bresenham
    canvas.h, 10
canvas_clear_pixel
    canvas.h, 10
canvas_draw
    canvas.h, 10
canvas_dump_set
    canvas.h, 11
canvas_free
    canvas.h, 11
canvas_init
    canvas.h, 11
canvas_print
    canvas.h, 12
canvas_set_pixel
    canvas.h, 12
clampi
    math_utils.h, 17
clampui
    math_utils.h, 17
clay_init
    ui_core.h, 19
clay_render
    ui_core.h, 19
clay_update_dimensions
    ui_core.h, 20
create_static_button
    ui_core.h, 20
cursor_in_bounds
    input.h, 15
Custom_Element_Data, 6
Custom_Element_Data::[union].__unnamed0__, 5

editor_layout
    ui_core.h, 21

input.h
    cursor_in_bounds, 15
    input_handler_init, 15
    KEY_JUST_PRESSED, 14
    KEY_JUST_RELEASED, 14
    KEY_PRESSED, 14
    KEY_RELEASED, 14
    keystate, 14
    update_key_state, 15
Input_Handler, 6
input_handler_init
    input.h, 15

KEY_JUST_PRESSED
    input.h, 14
KEY_JUST_RELEASED
    input.h, 14
KEY_PRESSED
    input.h, 14
KEY_RELEASED
    input.h, 14
keystate
    input.h, 14

math_utils.h
    clampi, 17
    clampui, 17

src/canvas.h, 9, 13
src/input.h, 13, 16
src/math_utils.h, 17, 18
src/ui_core.h, 18, 21

ui_core.h
    clay_init, 19
    clay_render, 19
    clay_update_dimensions, 20
    create_static_button, 20
    editor_layout, 21
update_key_state
    input.h, 15